Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Летучка № 3

по курсу «Методы оптимизации»

«Реализация методов первого порядка»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать следующие методы:

1. Метод градиентного спуска
2. Метод наискорейшего спуска
3. Метод сопряженных градиентов

Функции:

1. $f(x1, x2) = ax_1^2 + bx_2^2$
2. Растригна
3. Швеффеля
4. Розенброкка

2 Результаты

Исходный код программы представлен в листинге 1.

```
using LinearAlgebra
using Plots

function gd(f, grad_f, x0, alpha; max_iter=1000, tol=1e-8)
    x = copy(x0)
    traj = [copy(x)]
    for _ in 1:max_iter
        g = grad_f(x)
        if norm(g) < tol
            break
        end
        x = x - alpha * g
        push!(traj, copy(x))
    end
    return x, traj
end

function fib_sequence(n)
    F = ones(Int, n + 1)
```

```
    for i in 2:n
        F[i+1] = F[i] + F[i-1]
    end
    return F
end

function fibonacci_minimize(phi, a, b; n=25)
    F = fib_sequence(n)
    for k in n:-1:2
        r = b - a
        delta = r * F[k-1] / F[k+1]
        ap = a + delta
        bp = b - delta
        if ap >= bp
            break
        end
        if phi(ap) > phi(bp)
            a = ap
        else
            b = bp
        end
    end
    return (a + b) / 2
end

function bracket_interval(phi, alpha_max=1e10)
    a = 0.0
    b = 1.0
    phi0 = phi(a)
    while phi(b) < phi0 && b < alpha_max
        b *= 2
    end
    return a, min(b, alpha_max)
end

function steepest_descent(f, grad_f, x0; max_iter=1000, tol=1e-8,
    fib_n=25)
    x = copy(x0)
    traj = [copy(x)]
    for _ in 1:max_iter
        g = grad_f(x)
        if norm(g) < tol
            break
        end
        d = -g
```

```
        phi = alpha -> f(x + alpha * d)
        a, b = bracket_interval(phi)
        alpha = fibonacci_minimize(phi, a, b; n=fib_n)
        x = x + alpha * d
        push!(traj, copy(x))
    end
    return x, traj
end

function f_quad(a, b)
    x -> a * x[1]^2 + b * x[2]^2
end

function g_quad(a, b)
    x -> [2 * a * x[1], 2 * b * x[2]]
end

function f_rosenbrock(x)
    (1 - x[1])^2 + 100 * (x[2] - x[1]^2)^2
end

function g_rosenbrock(x)
    [-2 * (1 - x[1]) - 400 * x[1] * (x[2] - x[1]^2), 200 * (x[2] -
   x[1]^2)]
end

function f_schwefel(x)
    418.9829 * length(x) - sum(xi -> xi * sin(sqrt(abs(xi))), x)
end

function g_schwefel(x)
    g = similar(x)
    for i in eachindex(x)
        xi = x[i]
        s = sqrt(abs(xi)) + 1e-20
        if xi >= 0
            g[i] = -sin(s) - xi * cos(s) / (2 * s)
        else
            g[i] = -sin(s) + xi * cos(s) / (2 * s)
        end
    end
    return g
end

function f_rastrigin(x)
```

```
    10 * length(x) + sum(xi -> xi^2 - 10 * cos(2 * pi * xi), x)
end

function g_rastrigin(x)
    [2 * xi + 20 * pi * sin(2 * pi * xi) for xi in x]
end

function cg(f, grad_f, x0; max_iter=1000, tol=1e-8)
    x = copy(x0)
    n = length(x)
    g = grad_f(x)
    d = -copy(g)
    traj = [copy(x)]
    for k in 1:max_iter
        if norm(g) < tol
            break
        end
        alpha = 1.0
        while f(x + alpha * d) >= f(x) && alpha > 1e-15
            alpha /= 2
        end
        if alpha <= 1e-15
            break
        end
        x = x + alpha * d
        push!(traj, copy(x))
        g_new = grad_f(x)
        beta = (k % n == 0) ? 0.0 : dot(g_new, g_new) / (dot(g, g) + 1e-20)
        d = -g_new + beta * d
        g = g_new
    end
    return x, traj
end

x0 = [100.0, 100.0]
alpha = 0.1
cases = [(1, 1), (1, 5), (1, 10)]

all_x = Float64[]
all_y = Float64[]
for (a, b) in cases
    fq = f_quad(a, b)
    gq = g_quad(a, b)
    _, t1 = gd(fq, gq, x0, alpha; max_iter=2000)
```

```
    _, t2 = steepest_descent(fq, gq, x0; max_iter=2000)
    _, t3 = cg(fq, gq, x0; max_iter=2000)
    for t in (t1, t2, t3)
        for p in t
            push!(all_x, p[1])
            push!(all_y, p[2])
        end
    end
end
margin = 5.0
xmin = minimum(all_x) - margin
xmax = maximum(all_x) + margin
ymin = minimum(all_y) - margin
ymax = maximum(all_y) + margin

for (idx, (a, b)) in enumerate(cases)
    fq = f_quad(a, b)
    gq = g_quad(a, b)
    r = range(xmin, xmax, length=80)
    z = [fq([x, y]) for x in r, y in r]
    plt = contour(r, r, z, xlims=(xmin, xmax), ylims=(ymin, ymax))
    _, t1 = gd(fq, gq, x0, alpha; max_iter=2000)
    _, t2 = steepest_descent(fq, gq, x0; max_iter=2000)
    _, t3 = cg(fq, gq, x0; max_iter=2000)
    plot!(plt, [p[1] for p in t1], [p[2] for p in t1], label="", color=:blue)
    plot!(plt, [p[1] for p in t2], [p[2] for p in t2], label="", color=:red)
    plot!(plt, [p[1] for p in t3], [p[2] for p in t3], label="", color=:green)
    plot!(plt, xlabel="x1", ylabel="x2", title="f(x)=$(a)x1 +$(b)x2 ")
    savefig(plt, "quad_b$(b).png")
end

function clip_traj(traj, xmin, xmax, ymin, ymax)
    [(p[1], p[2]) for p in traj if xmin <= p[1] <= xmax && ymin <= p[2] <= ymax]
end

function plot_bench(f, g, x0, name, fname; xr=(-5.0, 5.0), yr=(-5.0, 5.0), alpha_plot=nothing)
    xmin, xmax = xr[1], xr[2]
    ymin, ymax = yr[1], yr[2]
    a = something(alpha_plot, alpha)
```

```
    r = range(xmin, xmax, length=80)
    z = [f([x, y]) for x in r, y in r]
    plt = contour(r, r, z, xlims=(xmin, xmax), ylims=(ymin, ymax), colorbar=true)
    _, t1 = gd(f, g, x0, a; max_iter=2000)
    _, t2 = steepest_descent(f, g, x0; max_iter=2000)
    _, t3 = cg(f, g, x0; max_iter=2000)
    c1 = clip_traj(t1, xmin, xmax, ymin, ymax)
    c2 = clip_traj(t2, xmin, xmax, ymin, ymax)
    c3 = clip_traj(t3, xmin, xmax, ymin, ymax)
    if length(c1) >= 2
        plot!(plt, [p[1] for p in c1], [p[2] for p in c1], label="", color=:blue, linestyle=:solid, linewidth=2.5)
    end
    if length(c2) >= 2
        plot!(plt, [p[1] for p in c2], [p[2] for p in c2], label="", color=:red, linestyle=:dash, linewidth=2.5)
    end
    if length(c3) >= 2
        plot!(plt, [p[1] for p in c3], [p[2] for p in c3], label="", color=:green, linestyle=:dot, linewidth=2.5)
    end
    plot!(plt, xlabel="x1", ylabel="x2", title=name)
    savefig(plt, fname)
end

x0_rosen = [-1.5, 2.0]
plot_bench(f_rosenbrock, g_rosenbrock, x0_rosen, "", "rosenbrock.png"; xr=(-3.0, 3.0), yr=(-2.0, 5.0), alpha_plot=0.001)

x0_schwefel = [400.0, 400.0]
plot_bench(f_schwefel, g_schwefel, x0_schwefel, "", "schwefel.png"; xr=(400.0, 425.0), yr=(400.0, 425.0))

x0_rastrigin = [4.0, 2.5]
plot_bench(f_rastrigin, g_rastrigin, x0_rastrigin, "", "rastrigin.png"; xr=(-5.0, 5.0), yr=(-5.0, 5.0))

for (a, b) in cases
    fq = f_quad(a, b)
    gq = g_quad(a, b)
    xg, _ = gd(fq, gq, x0, alpha)
    xh, _ = steepest_descent(fq, gq, x0)
```

```
    xc, _ = cg(fq, gq, x0)
    println("                                a=$a b=$b:")
    println("       : x = ", xg, ", f(x) = ", fq(xg))
    println("         : x = ", xh, ", f(x) = ", fq(xh))
    println("       : x = ", xc, ", f(x) = ", fq(xc))
end

x0 = x0_rosen
alpha_rosen = 0.001
xr_g, _ = gd(f_rosenbrock, g_rosenbrock, x0, alpha_rosen)
xr_h, _ = steepest_descent(f_rosenbrock, g_rosenbrock, x0)
xr_c, _ = cg(f_rosenbrock, g_rosenbrock, x0)
println("                    ,                                : ", x0)
println("       : x = ", xr_g, ", f(x) = ", f_rosenbrock(xr_g))
println("         : x = ", xr_h, ", f(x) = ", f_rosenbrock(xr_h))
println("       : x = ", xr_c, ", f(x) = ", f_rosenbrock(xr_c))

x0 = x0_schwefel
xs_g, _ = gd(f_schwefel, g_schwefel, x0, alpha)
xs_h, _ = steepest_descent(f_schwefel, g_schwefel, x0)
xs_c, _ = cg(f_schwefel, g_schwefel, x0)
println("                ,                                : ", x0)
println("       : x = ", xs_g, ", f(x) = ", f_schwefel(xs_g))
println("         : x = ", xs_h, ", f(x) = ", f_schwefel(xs_h))
println("       : x = ", xs_c, ", f(x) = ", f_schwefel(xs_c))

x0 = x0_rastrigin
xra_g, _ = gd(f_rastrigin, g_rastrigin, x0, alpha)
xra_h, _ = steepest_descent(f_rastrigin, g_rastrigin, x0)
xra_c, _ = cg(f_rastrigin, g_rastrigin, x0)
println("                    ,                                : ", x0)
println("       : x = ", xra_g, ", f(x) = ", f_rastrigin(xra_g))
println("         : x = ", xra_h, ", f(x) = ", f_rastrigin(xra_h))
println("       : x = ", xra_c, ", f(x) = ", f_rastrigin(xra_c))
```

Результат представлен на рисунках 1– 3.

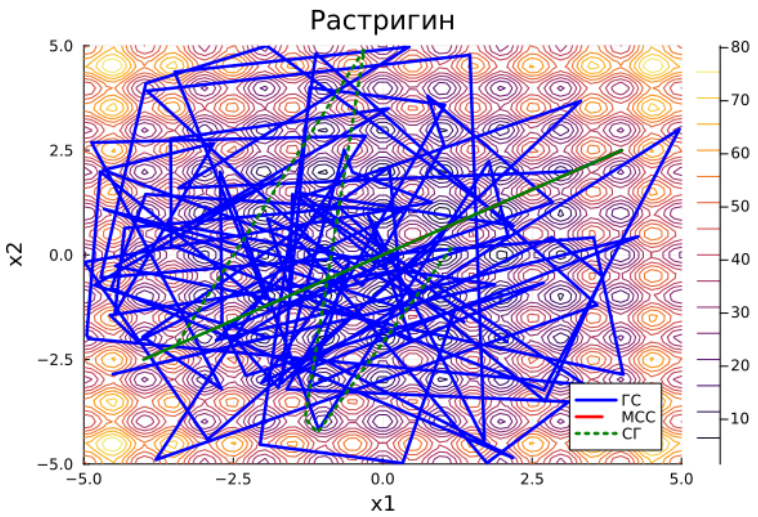

Рис. 1 — Результат

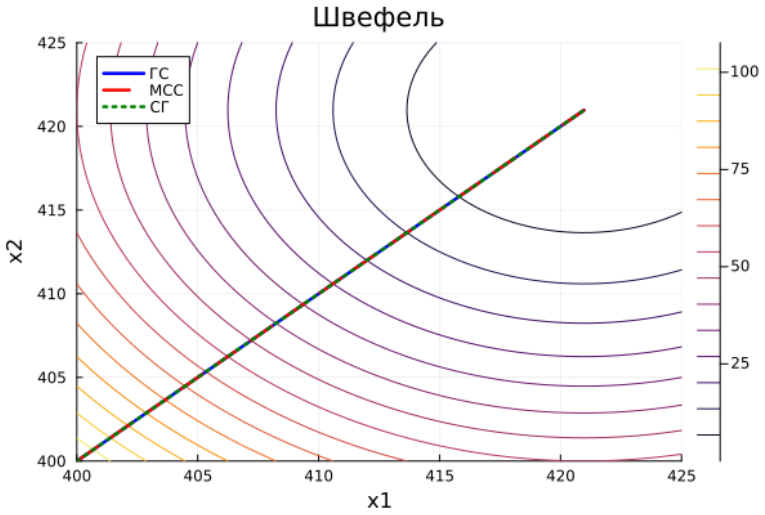

Рис. 2 — Результат

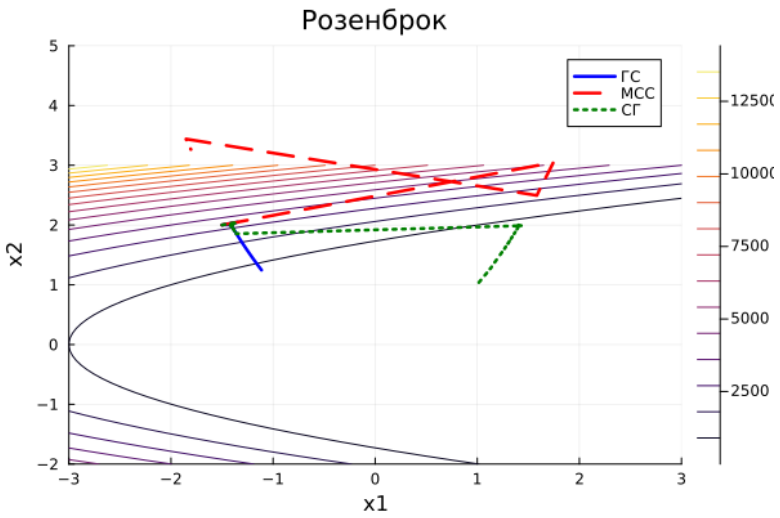

Рис. 3 — Результат